

Clean-Cooking Mini-Grid Tariff Simulator

Model Reference

How the agent-based day-ahead tariff simulator works, and every parameter value currently configured in `sim/config.py`.

Generated 2026-07-08

Population: 100 agents across personas household, school, kiosk
Meal menu: 18 meals (from `master_table_Z.md`)
Day resolution: 288 blocks of 5 minutes
Parameters: 51 total, 42 flagged as placeholder guesses (see the audit table)

Notation

Every symbol used in the equations on the following pages, in order of first use.

Symbol	Domain	Meaning
t	$\{0, \dots, T-1\}$	Discrete block index within one simulated day
$s(t)$	$\{B, L, D\}$	Active meal stage (breakfast/lunch/dinner) at time t (clock-time only)
$\mathbf{h}_t$	$\{0, \dots, K\}^3$	Meal-slot state (h_t^B, h_t^L, h_t^D); 0 = not yet eaten, else 1-based meal index
τ_t	$[0, \tau_{\max}]$	Hours since the agent last ate
n_t	$\{0, 1, 2, 3\}$	Meals eaten so far today, $n_t = \{s : h_t^s \neq 0\} $
$\bar{n}(t)$	$\{0, 1, 2, 3\}$	Step function: meals a schedule-following agent should have eaten by t
H_t	$\mathbb{R}_{\geq 0}$	Hunger
κ	scalar	Weight of τ_t in hunger
$\sigma(x)$	$(0, 1)$	Logistic sigmoid function -- squashes any real number into a probability
Δ	$(0, 1]$	Hazard-to-probability scale (also the max per-block probability)
ℓ_t	$\mathbb{R}$	Hazard logit (an unbounded score, not itself a probability)
$w(t)$	$\mathbb{R}$	Baseline hazard logit: time-of-day profile (3 Gaussian bumps)
η_t, η_k	$\mathbb{R}$	Scenario offsets: extra timing bias / extra meal-appeal
$\lambda_{p,s}$	$\mathbb{R}$	Per-(persona,stage) hazard bias -- shapes WHEN a persona cooks
α_0	scalar	Weight of hunger in the hazard logit
b	scalar	Deeply negative overnight floor $w(t)$ relaxes to, away from any meal
c_s	hour	Clock time where stage s 's hazard bump peaks
β_s	scalar	Peak value of $w(t)$ exactly at c_s -- how strong the pull is at that peak
σ_w	hours	Shared width of the 3 bumps (unrelated to the function $\sigma(\cdot)$ above)
q_t	$[0, \Delta]$	Probability the agent starts cooking in block t

Notation (cont.)

Symbol	Domain	Meaning
K	integer	Number of meals on the menu
$\mathbf{z}_k$	$\mathbb{R}^8$	Meal k 's fixed attribute vector, same for every agent (Table 2)
$\mathbf{y}$	$\mathbb{R}^8$	Agent's own weights on the 8 meal attributes -- shapes WHICH meal it picks
γ_{cost}	scalar	Agent's price sensitivity
$p(t)$	$\mathbb{R}_{\geq 0}$	Grid tariff price at time t (currency/kWh)
e_k	$\mathbb{R}_{\geq 0}$	Meal k 's grid energy draw (kWh); 0 for fire-only meals
α_k	$\mathbb{R}$	Meal k 's hunger-boost coefficient
u_k	$\mathbb{R}$	Meal k 's utility (meaningful only relative to other meals' u_j)
$P(k)$	$[0, 1]$	Probability of choosing meal k , conditional on firing
π	tariff	One candidate day-ahead tariff: a full price path $p(t)$, announced at $t = 0$
$\bar{p}$	currency/kWh	Common target average price every tariff candidate is rescaled to
p_{lo}, p_{hi}	currency/kWh	Low / high price levels used by the non-flat tariffs
W_{peak}	time window	Fixed evening window used by the evening_peak tariff (e.g. 17:00-21:00)
$PV(t)$	$\mathbb{R}_{\geq 0}$	Stylised solar-output curve; shapes solar_following only, not agent choices

The model

1. State space

Each agent is simulated as an independent discrete-time stochastic process over one day, divided into T blocks of Δt_{blk} minutes ($T = 288$ blocks of 5 minutes by default).

The agent's state at block t (an integer counting up from 0 at midnight) is the triple

$$s_t = (t, \mathbf{h}_t, \tau_t)$$

where $\mathbf{h}_t = (h_t^B, h_t^L, h_t^D) \in \{0, \dots, K\}^3$ is three numbers, one per meal stage (Breakfast/Lunch/Dinner): each is 0 if that meal hasn't been eaten yet today, or the 1-based index of whichever of the K menu items was eaten there. τ_t (tau) is simply the number of hours since the agent last ate anything at all, counting up from 0 every time it eats. Let $n_t = |\{s : h_t^s \neq 0\}|$ (a count from 0 to 3) be how many of the three meals the agent has already eaten today. Hunger, which drives both stages below, combines both ideas:

$$H_t = \max(0, \bar{n}(t) - n_t) + \kappa \tau_t$$

$\bar{n}(t)$ ($\bar{n}$ -bar) is a step function giving the number of meals an agent 'on schedule' should have eaten by clock time t (0 before breakfast, 1 after breakfast, 2 after lunch, 3 after dinner) -- so the first term is 0 while the agent is keeping pace, and only turns positive once it has fallen behind. κ (kappa) is a single fixed constant that adds a small, steadily growing amount of hunger for every hour since the last meal, on top of that -- so hunger never actually reaches zero for long, even for an agent that is technically 'on schedule'.

2. Stage 1 -- cooking hazard

At each block, if the active stage $s(t)$ (whichever of breakfast/lunch/dinner is open at the current clock time) still has an empty slot ($h_t^{s(t)} = 0$, i.e. the agent hasn't already eaten that meal), the agent begins cooking this one block with probability

$$q_t = \Delta \cdot \sigma(\ell_t), \quad \sigma(x) = \frac{1}{1 + e^{-x}}$$

The model (cont.)

$\sigma(x)$, the logistic sigmoid function, takes any real number x -- negative, positive, huge or tiny -- and squashes it into the open interval $(0, 1)$: very negative x gives a result near 0, very positive x gives a result near 1, and $x = 0$ gives exactly 0.5. It exists so that the unbounded score ℓ_t below can be turned into an actual probability. Δ (a fixed constant between 0 and 1, e.g. 0.15) then scales that probability down further and doubles as a hard ceiling: however hungry or however deep into a meal-time peak the agent is, q_t can never exceed Δ .

ℓ_t (a lower-case script L, read "the hazard logit") is the unbounded, arbitrary-scale score that $\sigma(\cdot)$ above converts into a probability -- it is not a probability itself, and can be any real number, very negative or very positive. It is built by adding up four separate influences, so each can be reasoned about on its own:

$$\ell_t = w(t) + \eta_t + \lambda_{p,s(t)} + \alpha_0 H_t$$

$w(t)$ (defined below) is a baseline pull toward cooking that depends only on the clock time t , the same for every agent regardless of persona. η_t (eta) is an optional, scenario-specific nudge to that baseline -- it is exactly 0 unless a non-default scenario is active. $\lambda_{p,s}$ (lambda) is a fixed number specific to both the agent's persona p (household, school or kiosk) and the currently-active stage s -- it does not depend on which meal is eventually chosen, only on who is cooking and when. This single number is how a whole persona's daily rhythm is shaped without writing separate code per persona: e.g. giving school a strongly negative λ at breakfast and dinner, and a strongly positive one at lunch, makes it fire almost exclusively around lunchtime, while household keeps every $\lambda_{p,s} = 0$ (no bias at any stage). Finally α_0 (alpha-zero, a single fixed constant shared by everyone) controls how strongly the hunger score H_t from Section 1 pushes up the logit -- and hence the firing probability -- as the agent gets hungrier.

$w(t)$ itself relaxes to a very negative overnight floor b away from three Gaussian-shaped "bumps", one per meal stage $s \in \{B, L, D\}$:

$$w(t) = b + \sum_s (\beta_s - b) \exp\left(-\frac{1}{2} \left(\frac{t - c_s}{\sigma_w}\right)^2\right)$$

The model (cont.)

b is a single, deeply negative constant (e.g. -6): far from every meal-time bump, $w(t) \approx b$, so ℓ_t is deeply negative, $\sigma(\ell_t)$ is essentially 0, and agents essentially never start cooking outside meal times. Each stage s then contributes one bump, described by three numbers: c_s is the clock time (in hours) at which that stage's pull is strongest -- e.g. $c_s = 12.5$ means the lunch pull peaks at 12:30. β_s (beta) is the actual value $w(t)$ reaches exactly at that peak time c_s -- so β_s directly sets how strong the pull to cook is at the busiest moment of that meal window (a bigger β_s means a bigger logit, and hence a higher firing probability, right at that time of day). And σ_w is a single width, shared by all three bumps, controlling how quickly that pull fades as t moves away from c_s in either direction -- a small σ_w gives a sharp, narrow peak, a large σ_w spreads the pull over a wider span of the day. (This σ_w is an unrelated constant to the sigmoid FUNCTION $\sigma(\cdot)$ above -- same Greek letter, different role: one is a fixed width, the other is a function applied to ℓ_t .) Away from all three peaks, the exponential term is essentially 0 and $w(t)$ returns to the floor b .

Current calibration: $b = -6$, $\sigma_w = 1.5\text{h}$; $(c_B, \beta_B) = (7.5\text{h}, 1.2)$,
 $(c_L, \beta_L) = (12.5\text{h}, 1.5)$, $(c_D, \beta_D) = (18.5\text{h}, 1.8)$; $\alpha_0 = 0.4$,
 $\kappa = 0.05$, $\Delta = 0.15$. Stage s may only fire inside its own clock-time window
(breakfast (5.0, 11.0), lunch (11.0, 16.0), dinner (16.0, 23.0), all in 24h clock hours).

3. Stage 2 -- meal choice

Conditional on firing, the agent chooses one meal $k \in \{1, \dots, K\}$ via a multinomial-logit (softmax) discrete-choice model: every meal k gets a utility score u_k , and meals with a higher utility are more likely (but never certain) to be chosen.

$$u_k = \boldsymbol{\gamma}^\top \mathbf{z}_k + \eta_k - \gamma_{\text{cost}} p(t) e_k + \alpha_k H_t$$

u_k has no inherent units or absolute meaning by itself -- only its size RELATIVE to the other meals' u_j matters, via the softmax formula below. $\mathbf{z}_k$ is meal k 's fixed attribute vector: 8 numbers -- taste, tradition, kid-acceptance, batch potential, ingredient cost, prep labour, kcal, fuel cost (Table 2) -- that describe the MEAL, and are the same for every agent. $\boldsymbol{\gamma}$ (gamma) is the opposite: it belongs to the AGENT, not the meal, and is also 8 numbers, one per attribute, saying how much this particular agent cares about each one. $\boldsymbol{\gamma}^\top \mathbf{z}_k$ ("gamma dot z sub k") is a dot product -- writing out both as 8 numbers each, it is just shorthand for the sum of each pair multiplied together:

The model (cont.)

$$\mathbf{y}^\top \mathbf{z}_k = \gamma_1 z_{k,1} + \gamma_2 z_{k,2} + \dots + \gamma_8 z_{k,8}$$

i.e. each of the meal's 8 attributes gets multiplied by how much this agent personally weights that attribute, and the 8 results are added together. For example, a large positive γ on tradition combined with a meal's high tradition score z_k adds a lot to u_k ; a negative γ on ingredient cost means a pricier meal (higher z_k on that attribute) SUBTRACTS from u_k instead. Every persona has its own mean $\mathbf{y}$ (Table 3), and every individual agent draws its own $\mathbf{y}$ once at initialisation as a small random perturbation around its persona's mean -- so agents of the same persona behave similarly but not identically.

η_k (eta) is a flat, scenario-specific bonus or penalty added to meal k 's utility -- zero in the default scenario, but e.g. a "festival" scenario could give a celebratory dish a positive η_k to represent extra festive appeal, independent of the agent's own

$\mathbf{y}$. γ_{cost} is a single non-negative number, specific to the agent, describing how price-sensitive it is; multiplying it by the current grid price $p(t)$ and by meal k 's grid energy draw e_k (in kWh), then SUBTRACTING that product from u_k , makes pricier grid electricity less attractive -- more so for agents with a larger γ_{cost} . $e_k = 0$ for fire-only meals, so $p(t)$ never affects their utility at all: this is the single mechanism by which the grid tariff can push demand toward or away from wood. Finally α_k (alpha sub k) adds extra utility to meal k in proportion to the agent's current hunger H_t (from Section 1) -- calibrated so more filling, higher-kcal meals get a bigger hunger boost than light ones.

$$P(k) = \frac{\exp(u_k)}{\sum_{j=1}^K \exp(u_j)}$$

$P(k)$ turns the K utility scores into an actual probability distribution over meals. Exponentiating each u_k turns DIFFERENCES in utility into RATIOS of likelihood -- a meal 2 utility-points better than another becomes about $e^2 \approx 7.4$ times more likely to be chosen, whatever the two absolute utility values happen to be. Dividing by the sum over every meal j then rescales the result so the K probabilities add up to exactly 1.

The model (cont.)

Current calibration: $\alpha_k = \text{ALPHA_SCALE} \times \text{kcal}_k / \text{KCAL_MAX}$, with $\text{ALPHA_SCALE} = 1$ (so a meal's hunger boost simply scales with how many kcal it has). ing_cost , prep_min , kcal and fuel-cost are each normalised to roughly $[0, 1]$ by dividing by a reference maximum ($\text{ING_COST_MAX_KES} = 150$, $\text{PREP_MIN_MAX} = 45$, $\text{KCAL_MAX} = 800$, $\text{CHARCOAL_KES_MAX} = 108$) every gamma-weighted feature sits on a comparable scale, and no single attribute dominates u_k just because of its raw units.

4. Demand and tariffs

Every cooking event contributes a kW profile (a boxcar of height e_k/D over its sampled duration D , by default) to the aggregate demand curve. A tariff π (pi) is simply a price path $p(t)$: one number per block over the whole day, announced in full at $t = 0$ (agents never see future prices before they're announced, but the whole path is fixed and known from block 0 onward, so there is no day-ahead uncertainty about price). Three candidate price paths are defined:

$$\text{flat: } p(t) = \bar{p}$$

$$\text{evening_peak: } p(t) = p_{lo} + (p_{hi} - p_{lo}) \cdot \mathbb{1}[t \in W_{peak}]$$

$$\text{solar_following: } p(t) = p_{hi} - (p_{hi} - p_{lo}) \cdot \frac{PV(t)}{\max_t PV(t)}$$

$\bar{p}$ (p-bar) is a single target price, the same for every candidate, that the whole tariff will average out to (see the rescaling below). p_{lo} and p_{hi} are a low and a high price level used by the two non-flat tariffs. $\mathbb{1}[t \in W_{peak}]$ is an indicator: it equals 1 for every block t inside the fixed evening window W_{peak} (e.g. 17:00-21:00) and 0 everywhere else, so evening_peak is cheap (p_{lo}) all day except that one expensive window. $PV(t)$ is a stylised solar-generation curve (a bell-shaped curve peaking at midday) used only to shape the solar_following price -- it has no other role and never feeds into any agent's decision directly; dividing by its own maximum $\max_t PV(t)$ rescales it to $[0, 1]$, so solar_following's price is cheapest when solar output is highest and most expensive when it's lowest. Every candidate is then rescaled by one constant factor so its own time-average equals $\bar{p}$ exactly -- this keeps the comparison about the SHAPE of the price over the day, not its overall level.

Personas

Household base gamma (every other persona's offsets are added on top of this):

```
taste      = +1.00
tradition  = +0.60
kid        = +0.40
batch      = +0.30
ing_cost   = -0.80
prep_min   = -0.40
kcal       = +0.20
fuelcost   = -0.80
gamma_cost = 1.2  (price sensitivity, shared by all personas)
sigma_ind  = 0.15 (individual noise std dev, sampled once per agent)
```

school (offsets on the household base; blank = inherits household unchanged):

```
taste      offset +0.30 -> effective +1.30
kid        offset +1.20 -> effective +1.60
batch      offset +1.50 -> effective +1.80
ing_cost   offset -1.50 -> effective -2.30
lam (per-stage hazard REPLACEMENT, not additive):
  breakfast=-6.0, lunch=+2.0, dinner=-6.0
```

kiosk (offsets on the household base; blank = inherits household unchanged):

```
taste      offset +1.40 -> effective +2.40
batch      offset +1.00 -> effective +1.30
prep_min   offset -0.80 -> effective -1.20
lam (per-stage hazard REPLACEMENT, not additive):
  (none -- household timing)
```

Population mix (agent counts, scaled to N_AGENTS):

```
household  80
school     10
kiosk      10
N_AGENTS   = 100
```

Meal menu (18 meals, from master_table_Z.md)

#	meal	type	min	kWh	kcal	ing.KES	taste	trad.	kid	batch	fire_only
1	uji_honey_sweetpotato	ELEC	45	0.75	280	25	0.5	0.8	1	0.4	
2	mahamri_mbaazi_zi_nazi	ELEC	60	1.2	780	60	0.8	0.4	0.9	0.8	
3	chai_roasted_sweetpotato_cassava	FIRE	45	0	360	30	0.55	0.85	0.8	0.3	Y
4	githeri_avocado_epc	ELEC	60	1	640	45	0.55	0.5	0.7	1	
5	ugali_ndengu_stew	ELEC	60	1.2	620	40	0.6	0.55	0.7	0.8	
6	ugali_sukuma_beef_stew	ELEC	90	2.25	800	90	0.8	0.6	0.8	0.6	
7	ugali_kuku_kienyeji_managu	ELEC	90	2.1	770	110	0.9	0.7	0.9	0.5	
8	chapati_maharagwe_ya_nazi	ELEC	90	2.4	680	55	0.85	0.5	0.95	0.7	
9	ugali_fried_tilapia_kachumbari	ELEC	45	1.13	730	100	0.85	0.5	0.6	0.2	
10	mukimo_beef_stew	ELEC	75	1.88	790	85	0.7	0.6	0.8	0.5	
11	matoke_beef	ELEC	45	0.9	420	60	0.6	0.5	0.7	0.5	
12	motori_bone_soup_cassava	ELEC	150	2	550	70	0.7	1	0.5	0.7	
13	nyama_choma_oven_kachumbari	ELEC	60	2	385	150	0.9	0.7	0.6	0.5	
14	nyama_choma_open_fire	FIRE	60	0	385	150	1	1	0.6	0.5	Y
15	tilapia_catfish_grilled_fire	FIRE	30	0	280	90	0.8	0.5	0.5	0.2	Y
16	motori_bone_soup_cassava_fire	FIRE	150	0	550	70	0.7	1	0.5	0.7	Y
17	ugali_sukuma_side	ELEC	45	1.13	410	30	0.6	0.6	0.8	0.3	
18	ugali_kachumbari_side	ELEC	30	0.75	320	20	0.55	0.6	0.8	0.3	

Parameter glossary -- state

block_minutes

value: 5.0 units: minutes
meaning: Length of one discrete simulation block.
effect: Smaller = finer time resolution, more blocks, slower run.

T

value: 288 units: blocks/day
meaning: Number of 5-minute blocks in a 24h day ($24 \times 60 / 5$).
effect: Derived from block_minutes; not an independent knob.

K

value: 18 units: meals
meaning: Size of the fixed meal menu (must match len(MEALS) below).
effect: Structural -- add/remove rows in the meal table to change this.

tau_cap_hr [TBD]

value: 24.0 units: hours
meaning: Cap on 'hours since last meal' (tau) so hunger doesn't grow unboundedly.
effect: Higher cap lets hunger (and firing pressure) keep climbing longer after a skipped meal.

n_stages

value: 3 units: stages
meaning: Number of meal stages per day: breakfast, lunch, dinner.
effect: Structural.

default_seed

value: 0 units: n/a
meaning: Default RNG seed for reproducibility when --seed is not given.
effect: Same seed + same config = identical output.

Parameter glossary -- timing

overnight_base_logit

value: -6.0 units: logit

meaning: Firing-hazard logit far from any meal bump (i.e. overnight).

effect: More negative = agents essentially never start cooking outside meal windows.

bump_centers_hr [TBD]

value: {'breakfast': 7.5, 'lunch': 12.5, 'dinner': 18.5} units: hour (24h clock)

meaning: Clock-time centre of each stage's cooking-hazard bump.

effect: Shifting a centre later moves that meal's typical start time later.

bump_heights [TBD]

value: {'breakfast': 1.2, 'lunch': 1.5, 'dinner': 1.8} units: logit

meaning: Peak hazard logit at the centre of each stage's bump (w(t) relaxes to overnight_base_logit away from the bump).

effect: Higher = sharper, more certain firing right at that meal's peak time.

bump_width_hr [TBD]

value: 1.5 units: hours

meaning: Gaussian width (std dev) of each meal-time hazard bump.

effect: Wider = firing probability spread over a longer window around the peak.

DELTA [TBD]

value: 0.15 units: probability/block (max)

meaning: Scale factor converting the hazard logit's sigmoid into a per-block firing probability; also the hard cap on that probability (sigmoid in [0,1]). Calibrated empirically (spec suggested 0.05 as a starting guess; that gave only ~37% of agents eating 3 meals/day under the reference tariff, so it was raised to 0.15, which gives ~90%, matching the spec's calibration target).

effect: The single most important tuning knob for how often anyone cooks at all -- raise it so more agents complete 3 meals/day, lower it for a sparser day.

Parameter glossary -- timing (continued)

stage_windows_hr [TBD]

value: {'breakfast': (5.0, 11.0), 'lunch': (11.0, 16.0), 'dinner': (16.0, 23.0)} units: hour (24h clock)

meaning: Clock-time window during which each stage's slot may fire at all; a stage cannot fire outside its window, and cannot fire again once its slot in h is nonzero.

effect: Narrower windows force meals into a tighter part of the day.

Parameter glossary -- hunger

nbar_step_times_hr [TBD]

value: {'0': 0.0, '1': 9.0, '2': 14.0, '3': 21.0} units: hour (24h clock)

meaning: Step curve nbar(t): expected cumulative meals eaten by this time of day (0 meals before 09:00, 1 after 09:00, 2 after 14:00, 3 after 21:00).

effect: Moving a step earlier makes agents 'behind schedule' (and hence hungrier) sooner.

kappa [TBD]

value: 0.05 units: hunger-units/hour

meaning: Weight converting hours-since-last-meal (τ) into hunger.

effect: Higher = time-since-eating dominates hunger; lower = only being behind on meal count matters.

alpha0 [TBD]

value: 0.4 units: logit/hunger-unit

meaning: Weight of hunger in the firing-hazard logit (Stage 1).

effect: Higher = hungrier agents become much more likely to start cooking at all.

Parameter glossary -- meals

ing_cost_max_kes [TBD]

value: 150.0 units: KES/serving

meaning: Reference max used to normalise ing_cost_kes into the ~0-1 gamma-weighted ing_cost feature (150 = the priciest meal in master_table_Z.md, nyama choma).

effect: Lower max makes ing_cost saturate (hit 1.0) for cheaper meals too, flattening the cost signal between mid-priced dishes.

prep_min_max [TBD]

value: 45.0 units: minutes

meaning: Reference max used to normalise prep_min into the ~0-1 gamma-weighted prep_min feature (45 = chapati, the most labour-intensive meal in the table).

effect: Lower max makes the labour penalty saturate for less demanding meals too.

kcal_max [TBD]

value: 800.0 units: kcal/serving

meaning: Reference max used to normalise kcal into the ~0-1 gamma-weighted kcal feature, and to scale ALPHA_SCALE-based hunger boosts (800 = ugali sukuma beef stew, the richest meal).

effect: Lower max makes both the kcal appeal term and the hunger boost saturate sooner.

charcoal_kes_max [TBD]

value: 108.0 units: KES/serving

meaning: Reference max used to normalise charcoal_kes into the fuelcost feature for fire-only meals (108 = the fire-only bone soup, the priciest charcoal cost).

effect: Lower max makes the fuelcost disincentive saturate for cheaper fire meals too.

alpha_scale [TBD]

value: 1.0 units: logit/hunger-unit (at kcal_norm=1)

meaning: Scales kcal_norm into each meal's alpha_k hunger boost ($\alpha_k = \text{ALPHA_SCALE} * \text{kcal}/\text{KCAL_MAX}$) -- richer meals satisfy hunger more, instead of hand-flagging individual meals as 'big'.

effect: Higher = kcal-rich meals become dramatically more attractive the hungrier an agent is.

Parameter glossary -- meals (continued)

meal_table [TBD]

value: '18 meals (see sim/config.py MEALS list)' units: n/a

meaning: The full 18-meal feature table transcribed from master_table_Z.md (Magadi/Kajiado West meal feature matrix): physical/cost columns (duration, kWh, charcoal kg/KES), nutrition (kcal, protein/carb/fat), and decision features (taste, tradition, kid-acceptance, batch potential, ingredient cost, prep labour). Individual meal values aren't tracked as separate glossary rows (300+ would swamp `explain`/`audit`) but the whole table is still a calibration placeholder.

effect: Per master_table_Z.md: 'All decision-column values are calibration placeholders -- priced at rough 2026 local market rates and scored by judgement. Replace with survey data (KAOP or household interviews) when available; the Z structure and lambda.z + gamma machinery don't change.'

duration_sigma_min [TBD]

value: 8.0 units: minutes

meaning: Std dev of per-cook duration draw $D \sim \text{Normal}(\text{Dbar}_k, \text{sigma})$.

effect: Higher = more variability in how long a cook (and its demand pulse) lasts.

duration_max_min [TBD]

value: 150.0 units: minutes

meaning: Hard clip on sampled cook duration.

effect: Prevents rare huge draws from producing absurdly long demand pulses.

profile_shape [TBD]

value: 'boxcar' units: n/a

meaning: Power-profile shape $\phi_k[j]$ used for every meal: 'boxcar' (flat kW for the whole cook, default) or 'preheat_simmer' (short high-power spike then a lower simmer level, same total energy). Plumbing for the latter exists; boxcar ships.

effect: Switching to preheat_simmer makes short-lived demand spikes sharper without changing daily household kWh.

Parameter glossary -- meals (continued)

preheat_spike_frac [TBD]

value: 0.2 units: fraction of duration

meaning: Fraction of a cook's duration spent in the preheat spike (preheat_simmer shape only).

effect: Larger = spike lasts longer (still same total energy, so simmer level drops).

preheat_power_mult [TBD]

value: 2.0 units: multiple of boxcar power

meaning: Preheat spike power as a multiple of the equivalent boxcar power (preheat_simmer only).

effect: Larger = sharper, taller spike at the start of the cook.

Parameter glossary -- personas

base_gamma [TBD]

value: {'taste': 1.0, 'tradition': 0.6, 'kid': 0.4, 'batch': 0.3, 'ing_cost': -0.8, 'prep_min': -0.4, 'kcal': 0.5}
meaning: Household base taste-weight vector (lambda in master_table_Z.md) applied to meal attributes z_k. First 7 features + values are transcribed directly from master_table_Z.md's 'house (base lambda)' column; fuelcost has no source-table row (the table doesn't list a gamma-weighted fuel-cost feature) so it reuses this project's earlier placeholder to keep fire meals non-free.
effect: ing_cost/prep_min should stay negative (they are costs); raising |ing_cost|/|prep_min| makes agents avoid expensive/demanding meals more strongly.

base_gamma_cost [TBD]

value: 1.2 units: utils per (currency/kWh * kWh)
meaning: Household price sensitivity: weight on -price(t)*e_k in the choice utility. THE only channel through which the grid tariff affects behaviour. Not part of master_table_Z.md (that table has no tariff/price-sensitivity concept); shared across all personas since the source table gives no override.
effect: The single most important knob for tariff response -- raise it and high prices push agents toward fire-only meals much more sharply.

sigma_ind [TBD]

value: 0.15 units: utils (std dev)
meaning: Std dev of per-agent individual variation around persona-mean gamma and gamma_cost, sampled once at init: $\gamma_i \sim \text{Normal}(\gamma_{\text{persona}}, \sigma_{\text{ind}}^2 * I)$.
effect: Higher = more heterogeneous population, less binary 'everyone defects at once' behaviour.

base_lam [TBD]

value: {'breakfast': 0.0, 'lunch': 0.0, 'dinner': 0.0} units: logit
meaning: Household per-stage firing-hazard bias lam[persona][stage], added into Stage 1's logit.
effect: Zero for the reference household persona (no stage is suppressed or boosted).

Parameter glossary -- personas (continued)

persona_gamma_offsets [TBD]

value: {'school': {'taste': 0.3, 'kid': 1.2, 'batch': 1.5, 'ing_cost': -1.5}, 'kiosk': {'taste': 1.4, 'batch': 1.5, 'ing_cost': -1.5}}

meaning: Group offsets transcribed from master_table_Z.md's 'school gamma' / 'kiosk gamma' columns: school upweights kid-acceptance and batch potential and is far more ingredient-cost averse (institutional budget); kiosk (mama ntilie / small food vendor) upweights taste even further and is much more prep-labour averse (cooks at commercial scale/pace, batches instead). Note: master_table_Z.md's prose says school 'downweights taste' but its own table lists school's taste offset as +0.3 (not negative) -- implemented per the table's number, flagging the discrepancy for review. Unspecified feature/persona pairs (tradition, kcal, fuelcost for both; ing_cost/kid for kiosk) get a 0 offset, i.e. inherit the household base unchanged.

effect: Larger |offset| = that persona diverges further from household on that one feature only.

persona_lam [TBD]

value: {'school': {'breakfast': -6.0, 'lunch': 2.0, 'dinner': -6.0}, 'kiosk': {}} units: logit (replacement)

meaning: Per-stage hazard-bias overrides: school's breakfast/dinner are switched off (-6, matching the overnight base logit) with a strongly boosted lunch (+2.0), making it essentially unimodal at lunch (one canteen meal/day). kiosk has no override at all (empty dict) -- master table Z.md gives no vendor operating-hours data, so it currently inherits household's flat, no-bias timing across all 3 stages. Revisit once real kiosk trading-hours data is available.

effect: An empty dict for a persona means 'same timing as household' -- not necessarily realistic for a vendor that might only trade at lunch/dinner.

mix [TBD]

value: {'household': 80, 'school': 10, 'kiosk': 10} units: agent count

meaning: Population composition across the three personas.

effect: More schools shifts the aggregate load curve toward a midday peak; more kiosks shift it toward whatever timing kiosk ends up with (currently household-like, see persona_lam).

Parameter glossary -- personas (continued)

N_AGENTS [TBD]

value: 100 units: agents

meaning: Total population size simulated per run.

effect: More agents = smoother aggregate demand curve, slower run.

Parameter glossary -- scenario

reference.name
value: 'reference' units: n/a
meaning: Default scenario: no timing or appeal offsets applied to anyone.
effect: This is the baseline every tariff sweep runs under unless --scenario overrides it.

reference.eta_t_hr_offsets
value: {} units: n/a
meaning: Scenario timing offset eta_t(t), reference case: identically zero everywhere.
effect: n/a

reference.eta_k
value: {} units: n/a
meaning: Scenario meal-appeal offset eta_k, reference case: identically zero for every meal.
effect: n/a

festival_day.name [TBD]
value: 'festival_day' units: n/a
meaning: Example alternative scenario demonstrating eta portability: a festival raises the appeal of big/traditional meals and pushes dinner later.
effect: Included as a template for adding new scenarios; not used unless --scenario festival_day.

festival_day.eta_t_hr_offsets [TBD]
value: {'dinner': {'centre_shift_hr': 1.5, 'height': 1.0, 'width_hr': 1.0}} units: hour / logit / hour
meaning: Adds an extra hazard bump 1.5h after the normal dinner centre, height 1.0, width 1h -- on top of the base dinner bump, biasing firing later in the evening.
effect: Larger centre_shift_hr / height = dinner drifts later / more sharply.

festival_day.eta_k [TBD]
value: {'nyama_choma_oven_kachumbari': 0.5, 'nyama_choma_open_fire': 1.0} units: utils
meaning: Flat utility bonus added to the two nyama_choma (grilled meat) options -- the classic Kenyan celebration dish -- biggest for the traditional open-fire one, representing festival appeal.
effect: Raises the odds those meals are chosen conditional on firing.

Parameter glossary -- tariff

p_bar [TBD]

value: 0.25 units: currency/kWh

meaning: Common time-average price every candidate tariff is normalised to.

effect: Scales all tariffs up/down together; relative shape (peak/flat/solar) is what's compared.

p_lo [TBD]

value: 0.1 units: currency/kWh

meaning: Off-peak / low price level used by evening_peak and solar_following.

effect: Lower = cheaper off-peak incentive to shift load away from the peak/evening.

p_hi [TBD]

value: 0.45 units: currency/kWh

meaning: Peak price level used by evening_peak and solar_following.

effect: Higher = stronger price signal to avoid cooking electrically at peak times -- more wood defection.

w_peak_hr [TBD]

value: (17.0, 21.0) units: hour (24h clock)

meaning: Evening peak window for the evening_peak tariff.

effect: Wider window = longer stretch of expensive electricity.

pv_p_max_kw [TBD]

value: 40.0 units: kW

meaning: Peak PV output used only to shape the solar_following tariff (never feeds agent utility).

effect: Larger = solar_following's price trough is based on a bigger notional solar fleet.

pv_clearness [TBD]

value: 0.8 units: fraction

meaning: Sky clearness factor scaling the PV stub's output.

effect: Lower = cloudier reference day, less pronounced solar-following price dip.

Parameter glossary -- tariff (continued)

pv_t_rise_hr [TBD]

value: 6.5 units: hour (24h clock)

meaning: Sunrise time used by the PV stub.

effect: Later sunrise shrinks the window where solar_following prices are cheap.

pv_t_set_hr [TBD]

value: 18.5 units: hour (24h clock)

meaning: Sunset time used by the PV stub.

effect: Earlier sunset shrinks the window where solar_following prices are cheap.

cap_kw [TBD]

value: 30.0 units: kW

meaning: Grid/mini-grid aggregate demand cap used for the exceedance penalty.

effect: Lower cap = exceedance probability rises for the same population -> tariffs scored harsher.

Parameter glossary -- scoring

PI [TBD]

value: 5.0 units: score/probability

meaning: Penalty weight on P_{exceed} in $\text{score} = \text{wood_share} + \text{PI} * P_{\text{exceed}}$.

effect: Higher = the scoreboard cares much more about avoiding cap breaches than about wood share.

R [TBD]

value: 200 units: runs

meaning: Number of independent Monte Carlo days simulated per tariff.

effect: More runs = more stable P_{exceed} estimate (spec target: stable to ± 0.02 across seeds), slower sweep.